

C++ Stack

Stacks in C++ programming language play an important role in LIFO (Last in first out) context which means elements are inserted and extracted only from one end.

C++ Stack Declaration

```
stack < datatype > stack_Name
```

For example,

```
stack < int > Stack1,
```

List of functions associated with Stack

- **push(g):** This function will add/insert the element or data value 'g' at the top of the stack.
- **pop():** This function will remove/delete the element or data value at the top of the stack.
- **top():** This function will return a reference to the topmost element of the stack.

- **size():** This function will return the size of the stack container that is a total number of elements stored in the stack.
 - **empty():** This function will check whether the stack container is empty or not.
-

Example1: Stack in C++

```
#include <iostream>
#include <stack>
using namespace std;

void stackPrint ( stack < int > ); // Prototype of Function that accept Stack as parameter

int main ()
{
    stack < int > stack1; //Declare a Stack
    stack1.push ( 100 ); // Insert element to the stack
    stack1.push ( 200 );
    stack1.push ( 300 );
    stack1.push ( 400 );
    cout << "Stack1 is consists of these numbers: ";
    stackPrint ( stack1 ); // call the Function stackPrint to display stack
    cout << "\n stack1.size() : " << stack1.size(); // get the size of stack
    cout << "\n stack1.top() : " << stack1.top(); // get the Top element of stack
    cout << "\n stack1.pop() : ";
    stack1.pop(); //print the Top element
    stackPrint( stack1 ); //call the Function stackPrint to display stack after removing the Top
    return 0;
}

void stackPrint ( stack < int > stk) // Function to display all the stack elements
{
    while ( !stk.empty() ) //while stack is not empty
    {
        cout << '\t' << stk.top(); // print the top then delete it (to avoid endless loop)
        stk.pop();
    }
}
```

```
Stack1 is consists of these numbers: 400 300 200 100
stack1.size() : 4
stack1.top() : 400
stack1.pop() : 300 200 100
```

Exercises:

- 1) Create a Stack of size 10 , ask the user to push the elements, pop twice then print on screen the content of stack .
- 2) Modify a simple program that ask a user to enter numbers , then display only even numbers.
- 3) Write a code that return true if the user write a character, and false if write an integer.